

Measuring Effectiveness

Important to answer this:
How Scalable is a Parallel Program?

Speedup and Efficiency

See section 0.3 in PDC For Beginners

Speedup:

$$S_p = T_s / T_p$$

T_s = time for sequential program

Time of sequential version gives us true speedup

We could use time of parallel version running on one core

Gives us relative speedup

T_p = time for parallel program

Code for parallel loop, reduction patterns

```
#define SIZE 67000000

int main(int argc, char** argv) {
    int array[SIZE];

    if (argc > 1) {
        omp_set_num_threads( atoi(argv[1]) );
    }
    double start = omp_get_wtime(); // wall clock time

    initialize(array, SIZE);
    double total = parallelSumSqrt(array, SIZE);

    // Measuring the elapsed time
    double end = omp_get_wtime();
    // Time calculation (in seconds)
    double elapsed_time = end - start;
    printf("Parallel time: %lf seconds\n", elapsed_time);

    return 0;
}
```

```
/* fill array with random values */
void initialize(double* a, int n) {
    int i;
    for (i = 0; i < n; i++) {
        a[i] = (double) (rand() % 100000);
    }
}
```

```
/* sum the array using multiple threads */
double parallelSumSqrt(double* a, int n) {
    double sum = 0.0;
    int i;
    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < n; i++) {
        sum += sqrt(a[i]);
    }
    return sum;
}
```

not
parallel

parallel

Suppose executing this program with one thread takes 2 seconds.

T_s sequential time

Using 2 threads should take about 1 seconds (ideally)

T_p parallel time, where $p = 2$

Using 4 threads should take about 0.5 seconds (ideally)

T_p parallel time, where $p = 4$

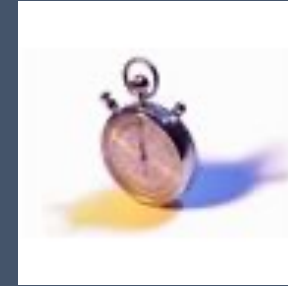
and so on- doubling the threads each time should cut the time in half

Speedup for multicore with OpenMP

Number of cores = p

Serial run-time = $T_{\text{serial}} = T_s$

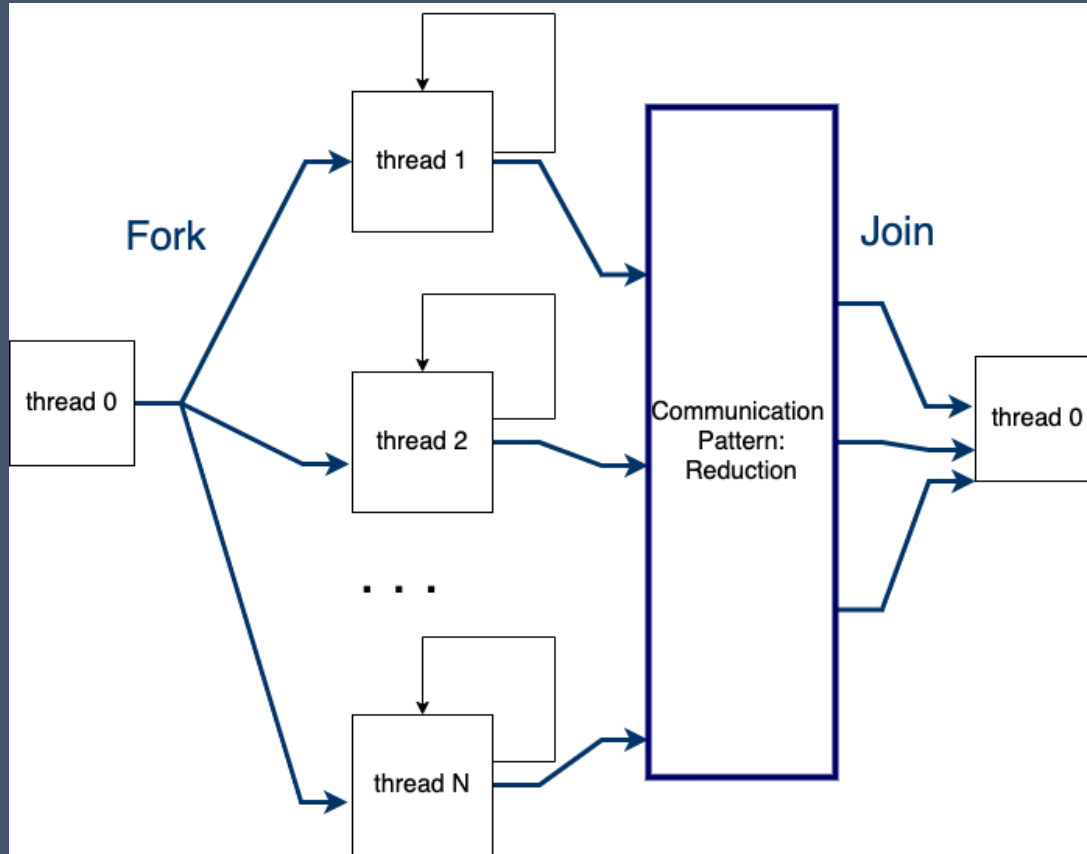
Parallel run-time = $T_{\text{parallel}} = T_p$



linear speedup

$$T_{\text{parallel}} = T_{\text{serial}} / p$$

However, there is overhead in parallel programs



```
#pragma omp parallel for reduction(+:sum)
for (i = 0; i < n; i++) {
    sum += sqrt(a[i]);
}
```

Fork, reduction, and Join take time.

If there is enough work going on in the loops on each thread, this time will be dwarfed by that work.

If not, we will be far from ideal speedup.

Speedup and Efficiency

Speedup, $S_p = T_s / T_p$

$S_2 = 2 / 1$ in the ideal 'linear' situation from previous code

Efficiency helps us measure the overhead

$$E_p = S_p / p = T_s / (p * T_p)$$

$E_2 = S_2 / 2 = 1$, or 100% in the ideal situation from previous code

Efficiency near 1, or 100% is ideal; usually above 75% is fairly good

p is number of threads used in a parallel OpenMP program

You can also think of the p subscript as saying Efficiency of the parallel program

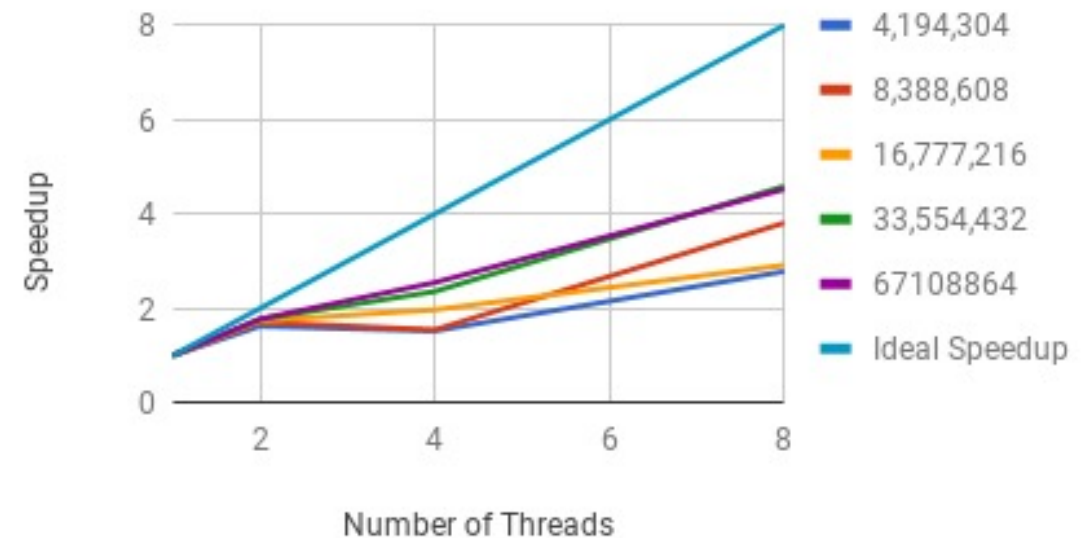
How we experiment and report speedup

	Array size				
Time for threads	4,194,304	8,388,608	16,777,216	33,554,432	67108864
1	0.140389	0.278735	0.554305	1.10265	2.190898
2	0.086532	0.162967	0.318345	0.623389	1.233185
4	0.092176	0.180323	0.281316	0.467516	0.859808
8	0.050442	0.073141	0.190127	0.240296	0.484111

Speedup $S_p = T_s / T_p$

For threads	4,194,304	8,388,608	16,777,216	33,554,432	67108864	Ideal Speedup
1	1	1	1	1	1	1
2	1.622	1.710	1.741	1.769	1.777	2
4	1.523	1.546	1.970	2.359	2.548	4
8	2.783	3.811	2.915	4.589	4.526	8

Speedup

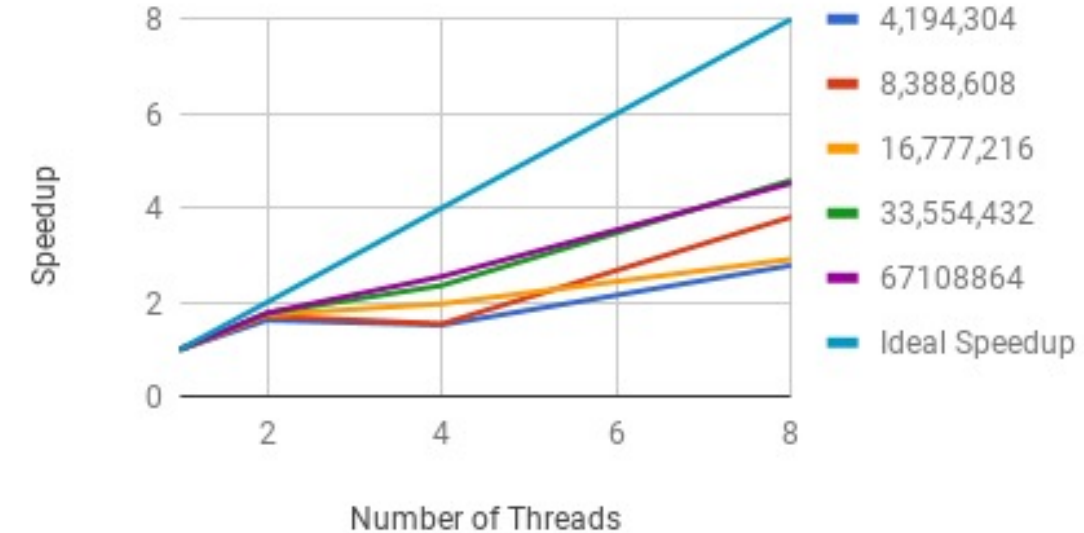


How we experiment and report speedup

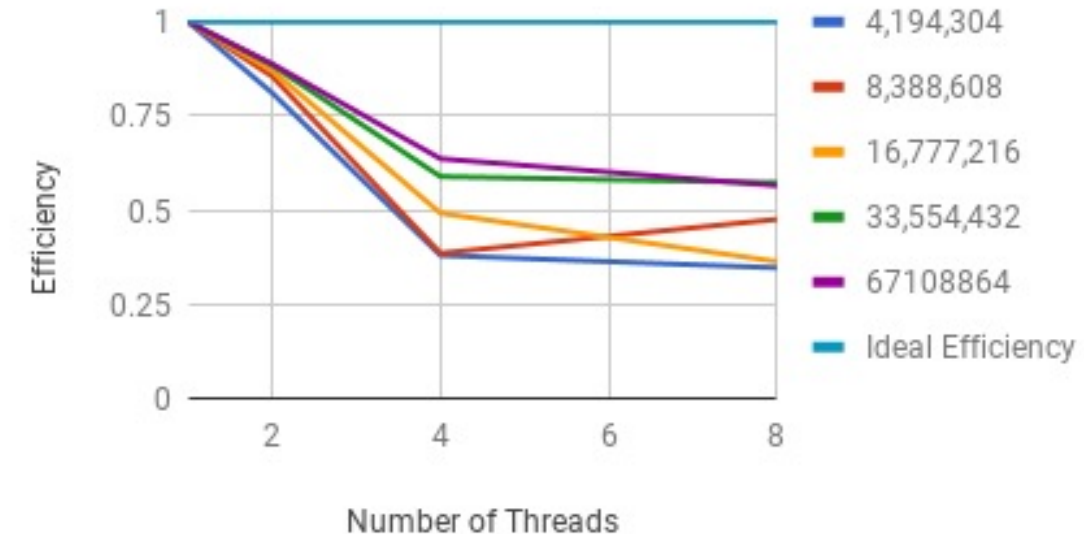
Speedup $S_p = T_s / T_p$						
For threads	4,194,304	8,388,608	16,777,216	33,554,432	67108864	Ideal Speedup
1	1	1	1	1	1	1
2	1.622	1.710	1.741	1.769	1.777	2
4	1.523	1.546	1.970	2.359	2.548	4
8	2.783	3.811	2.915	4.589	4.526	8

Efficiency $= S_p / p$						
	4,194,304	8,388,608	16,777,216	33,554,432	67108864	Ideal Efficiency
1	1	1	1	1	1	1
2	0.811	0.855	0.871	0.884	0.888	1
4	0.381	0.386	0.493	0.590	0.637	1
8	0.348	0.476	0.364	0.574	0.566	1

Speedup



Efficiency



Scalability

Speedup near ideal, or efficiency above 75%, for a particular problems size and number of threads, demonstrates *strong scalability*

Another form of scalability that is poorly named is *weak scalability*

if we increase the size of a problem by some factor

and we increase the number of threads by that factor

if the time remains the same or close to it, we have weak scalability

This kind of scalability can be very useful:

we can solve a larger problem by using a larger machine

with more cores

Scalability

In general, a problem is *scalable* if it can handle ever increasing problem sizes.

If we increase the number of processes/threads and keep the efficiency *roughly* fixed without increasing problem size, the problem is *strongly scalable*.

If we can keep the efficiency *roughly* fixed by increasing the problem size at the same rate as we increase the number of processes/threads, the problem is *weakly scalable*.

I emphasize the words *roughly* in the two definitions above, because ideal speedup is rarely possible

This example?

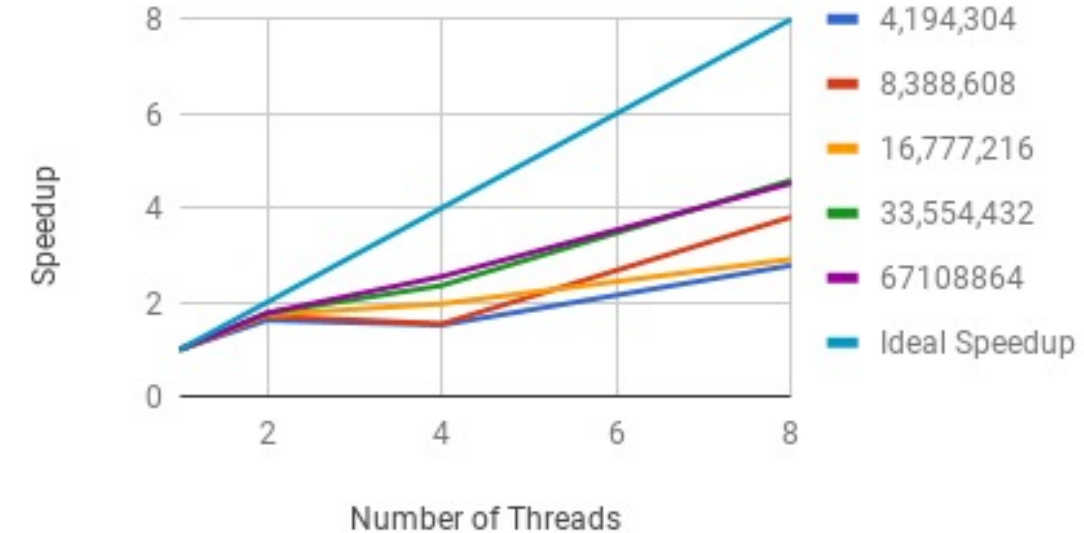
Strongly scalable?

Is speedup near ideal?

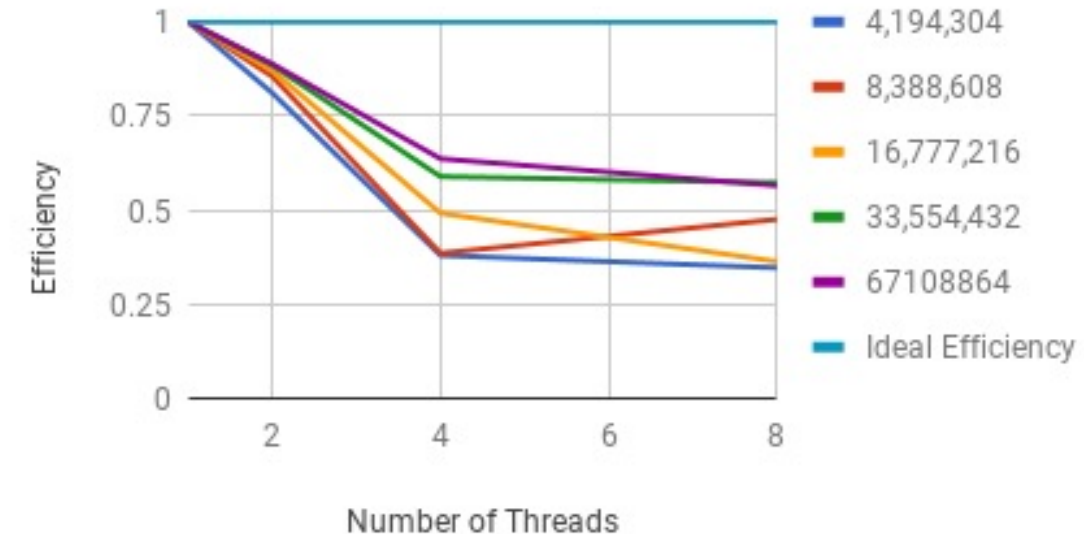
When is efficiency above 75%?

Would not characterize as strongly scalable *at these problem sizes*.

Speedup



Efficiency

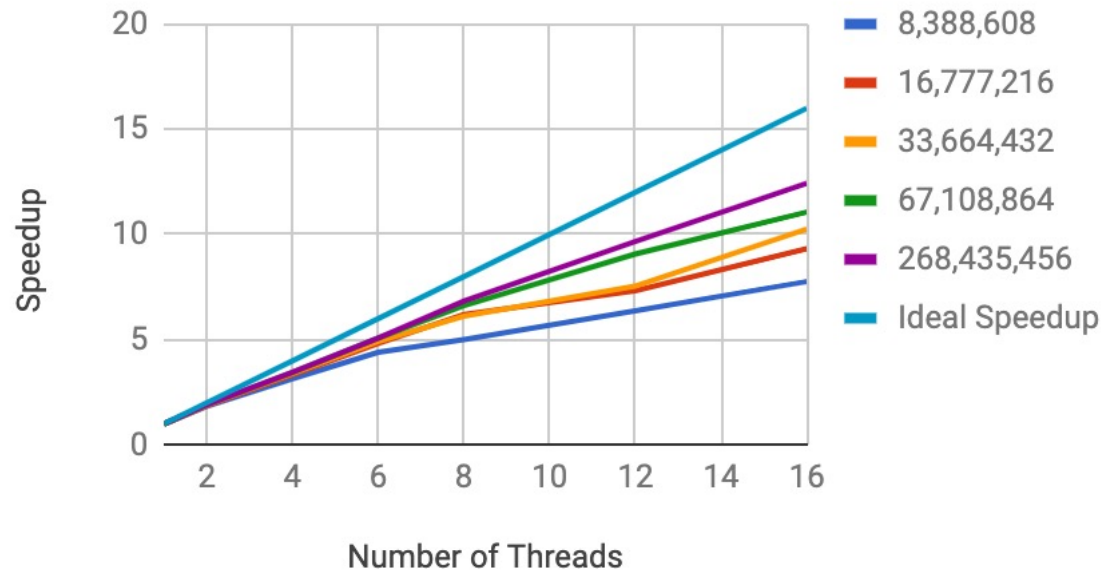


How about this in terms of strong scalability?

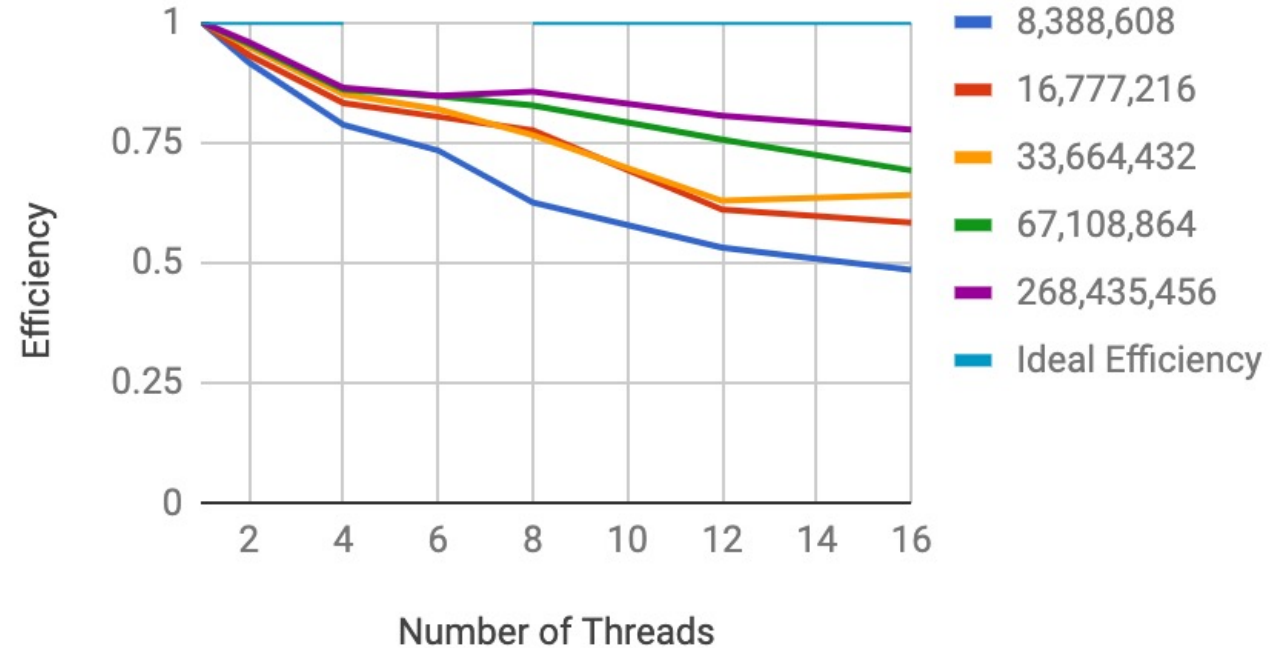
Is speedup near ideal?

When is efficiency above 75%?

Speedup

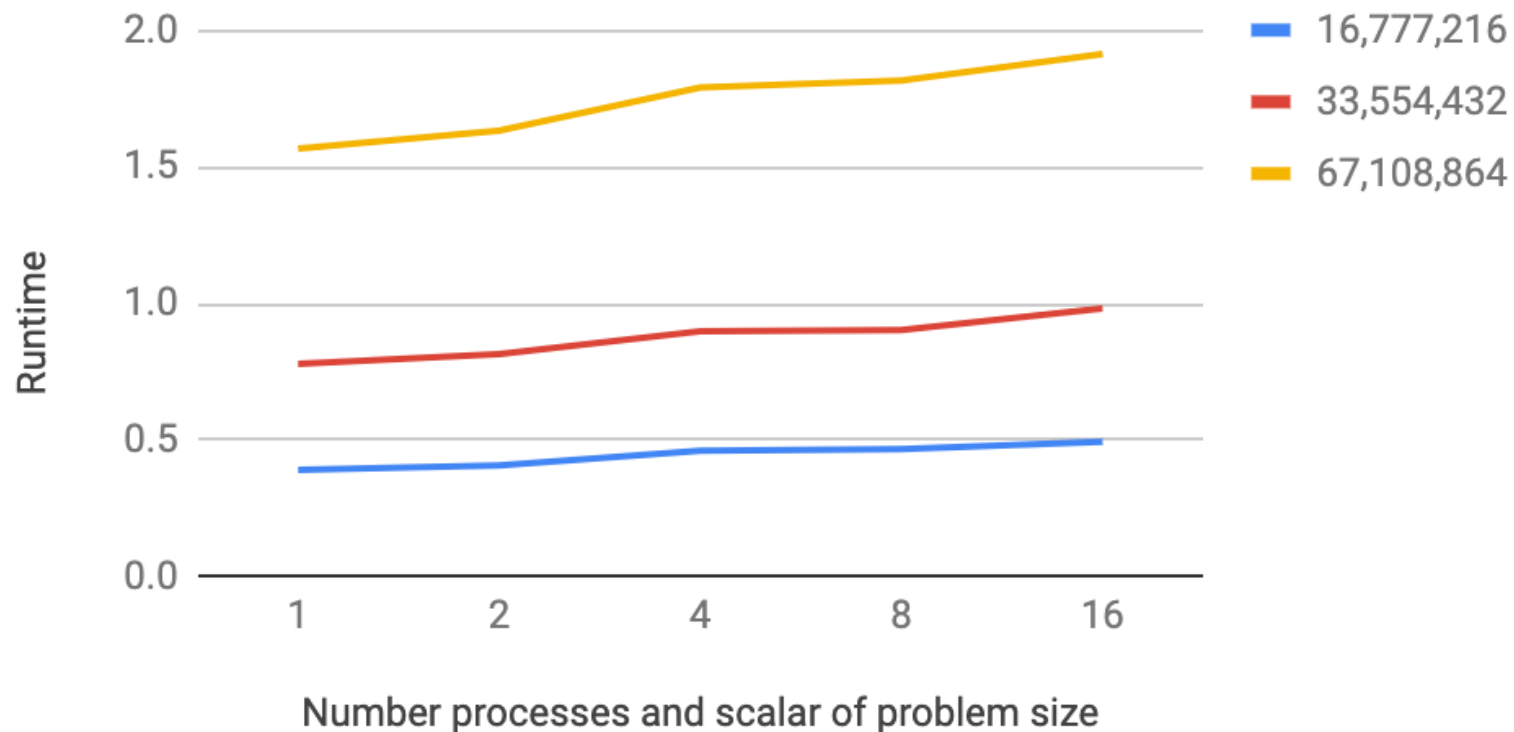


Efficiency



Is this weak scalability?

Weak Scalability: Scaling the number of processes and the problem size proportionally by scalar of 2



If we can keep the efficiency *roughly* fixed (i.e. has same time) by increasing the problem size at the same rate as we increase the number of processes/threads, the problem is *weakly scalable*.

Experimental process for this course

Use 'shell scripts' to execute code several times

- duplicate experiments

- run several different conditions

Transfer data to spreadsheet

- analyze using different measurements

 - speedup and efficiency

 - for strong scalability

 - run times as problem size and number of threads are increased proportionally

 - for weak scalability

Sequential timing

```
#include <omp.h>
. . .

double start = omp_get_wtime();

// compute each rectangle, adding
// area to the accumulator
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

// Measuring the elapsed time
double end = omp_get_wtime();
```

We use this for sequential timing too:

- Easy to use,
- Can compare times properly

Parallel timing

```
#include <omp.h>
. . .
// for timing
double start = omp_get_wtime();

// compute each rectangle, adding area to
// the accumulator
#pragma omp parallel for default(none) \
private(i) shared(n, a, h)
reduction(+:integral)
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

// Measuring the elapsed time
double end = omp_get_wtime();
. . .
```